

Preparación del examen oral — Advent of Code 2025

Principios y diseños de Ingeniería del Software II

Días 1 y 2

Índice

Cómo usar esta guía	1
Bloque 1 — Preguntas tipo (respuesta breve)	2
Arquitectura por capas	2
DIP — Inversión de Dependencias	2
OCP — Abierto/Cerrado	2
SRP — Responsabilidad Única	2
Strategy	2
Builder + Fluent API	2
Records e inmutabilidad	3
ISP, DI y composition root	3
Cohesión, acoplamiento, DRY	3
Bloque 2 — Preguntas difíciles (simulacro, respuesta modelo)	3
Cierre: frase para abrir la defensa	5

Cómo usar esta guía

Cada pregunta parte de un principio o patrón y te pide **señalarlo en tu propio código**. Responde siempre igual: di el principio en una frase, nómbralo en una clase concreta de tu AoC, y explica qué pasaría si lo quitaras. El examinador valora que apuntes al código, no que recites la teoría.

Mapa rápido de tus clases por capa:

model	Dial, Instruction, SecurityProtocol		IdRange
io	InstructionLoader		RangeLoader
control	Safe, SecurityProtocols		Engine, EngineBuilder, ValidationStrategie
application	ResourceInstructionLoader, Main01a/b		ResourceRangeLoader, Main02a/b

Bloque 1 — Preguntas tipo (respuesta breve)

Arquitectura por capas

¿Por qué divides cada día en model, io, control y application? Para separar responsabilidades y que las dependencias apunten siempre hacia el dominio. model no importa nada del proyecto; es el centro estable.

Dibuja la dirección de dependencias y di quién no puede depender de quién. application → control → (io + model) y io → model. El dominio nunca depende de las capas externas.

Si te pido leer de una base de datos en vez de un fichero, ¿qué clases tocas? Solo creo otra implementación de InstructionLoader/RangeLoader en application. El control y el model no se tocan.

DIP — Inversión de Dependencias

¿Dónde aplicas DIP? Safe depende de la abstracción SecurityProtocol, no de la lógica de la Parte A/B. En el día 2, Engine depende de LongPredicate. Y el arranque depende de la interfaz InstructionLoader/RangeLoader, no del detalle de lectura.

¿Quién depende de quién: la caja fuerte de la regla, o la regla de la caja fuerte? Ambas dependen de la abstracción. La regla concreta (SecurityProtocols) implementa la interfaz que vive en el dominio.

OCP — Abierto/Cerrado

Imagina una Parte C con una regla nueva. ¿Modificas Safe? No. Creo otra constante en SecurityProtocols y la inyecto en un nuevo Main. Safe queda cerrada a modificación y abierta a extensión por polimorfismo.

SRP — Responsabilidad Única

Dame una clase y su única razón para cambiar. Dial: solo si cambia la aritmética circular. Instruction: solo si cambia el formato de la orden. ValidationStrategies: solo si cambia la definición de “inválido”.

¿Qué refactor hiciste pensando en SRP? Saqué el parseo de texto fuera de Safe a Instruction.of, y separé la lectura del ensamblaje en el día 2 (antes EngineBuilder leía y construía; ahora solo construye y ResourceRangeLoader lee).

Strategy

¿Dónde tienes un Strategy? SecurityProtocol (día 1) y LongPredicate con ValidationStrategies (día 2): la regla es un algoritmo intercambiable que se inyecta.

Builder + Fluent API

¿Dónde tienes una Fluent API? En EngineBuilder: new EngineBuilder().from(ranges).use(strat

¿Por qué es “fluida” y cómo se consigue técnicamente? Porque encadeno llamadas en una sola expresión. Se logra porque `from()` y `use()` devuelven `this`, así cada método deja el objeto listo para la siguiente llamada.

¿Ventaja del Builder frente a un constructor con muchos parámetros? Construyo paso a paso, con nombres legibles, y valido que esté completo antes de crear (`runner()` lanza excepción si falta `from` o `use`), evitando objetos a medio construir.

Records e inmutabilidad

¿Por qué Dial es un record immutable? Es un valor del dominio sin identidad; `rotate()` devuelve un `Dial` nuevo en vez de mutar. Evita efectos colaterales y es seguro de compartir.

¿Dónde normalizas la posición 0-99 y por qué ahí? En el constructor compacto del record: así es imposible crear un `Dial` fuera de rango (encapsulo el invariante).

ISP, DI y composition root

¿Por qué los Loader tienen un solo método? ISP: una interfaz pequeña y cohesiva; nadie depende de operaciones que no usa.

¿Qué es el composition root en tu proyecto? Los Main: el único sitio donde elijo las piezas concretas (cargador + estrategia) y las conecto por inyección.

Cohesión, acoplamiento, DRY

Ejemplo de alta cohesión. `Dial` solo hace matemáticas circulares; `IdRange` solo entiende de rangos.

Ejemplo de DRY. La lectura de entrada está centralizada en una sola implementación, no repetida en cada Main.

Bloque 2 — Preguntas difíciles (simulacro, respuesta modelo)

1. Diferencia entre DIP y DI. ¿No son lo mismo? No. **DIP** es un principio de diseño: los módulos de alto nivel y los de bajo nivel dependen ambos de una abstracción, y la abstracción no depende de los detalles. **DI** (inyección de dependencias) es la **técnica** concreta con la que cumplo ese principio: paso la dependencia desde fuera en vez de crearla dentro. En mi código, el DIP es que `Safe` conoce `SecurityProtocol` (abstracción); la DI es que esa estrategia entra por el constructor desde el Main. Puedo cumplir DIP con DI por constructor, por setter, etc.

2. Defiéndeme por qué SecurityProtocol está en model y no en control, donde están sus implementaciones. Porque la interfaz solo habla el idioma del dominio: su firma es `calculatePoints(Dial, int, Dial)`, no depende de control, ni nada externo. La regla “cómo se puntúa un giro” es un concepto del dominio. Las **implementaciones concretas** (`SecurityProtocols`, que ya deciden A o B) sí son control. Poner la abstracción en el dominio es lo que hace que las flechas apunten hacia dentro: control depende de

model, nunca al revés. Es defendible ponerla en control junto a sus implementaciones, pero perdería esa señal de que la regla forma parte del modelo.

3. ¿Tu Engine cumple LSP? ¿Dónde podría romperse? LSP dice que cualquier implementación de una abstracción debe poder sustituir a otra sin romper al cliente. En el día 2 mis estrategias son LongPredicate: cualquier predicado encaja en Engine sin que este note la diferencia. Se rompería LSP si una implementación cambiara el contrato esperado: por ejemplo, una estrategia que lanzara excepción para números válidos, o que tuviera efectos colaterales, o que solo funcionara para ciertos rangos. Mientras todas respeten “recibo un long, devuelvo si es válido, sin sorpresas”, LSP se mantiene.

4. Tu Engine es un record que guarda un Stream. ¿Ves algún riesgo? Sí, y es un buen matiz: un Stream es de **un solo uso**. Si llamara a run() dos veces sobre el mismo Engine, la segunda lanzaría IllegalStateException porque el stream ya se consumió. En mi flujo no pasa porque cada Main construye el Engine y llama run() una vez. Si quisiera reutilizarlo, el diseño correcto sería guardar una List<IdRange> (o un Supplier<Stream<...>>) en vez del stream ya abierto. Lo reconozco como una limitación consciente del diseño actual.

5. ¿Por qué Instruction.of devuelve Optional y no lanza una excepción o devuelve null? Porque una línea inválida (nula, vacía, basura) no es un error excepcional: es un caso normal de entrada sucia que quiero **filtrar**, no abortar. Optional lo expresa en el tipo: el llamador está obligado a tratar el “puede no haber instrucción”. null no obliga a nada y propaga NullPointerException; una excepción mezclaría control de flujo con validación. Así Safe.rotate queda en Instruction.of(s).ifPresent(this::apply) y el cargador en .flatMap(Optional::stream).

6. ¿Builder y Factory no hacen lo mismo? ¿Por qué Builder aquí? Una Factory decide *qué* objeto crear y lo devuelve en un paso. Un **Builder** construye *un* objeto complejo *paso a paso*, acumulando configuración antes de materializarlo. Uso Builder en EngineBuilder porque el Engine necesita dos piezas (fuente y estrategia) que quiero fijar por separado, con interfaz fluida y una validación final que garantiza que no se cree incompleto. Una Factory tendría sentido si solo eligiera entre variantes ya cerradas.

7. Distingue acoplamiento aferente y eferente en tu día 1. El acoplamiento eferente (salida) de Safe es bajo: depende de Dial, Instruction y SecurityProtocol, todas abstracciones o valores del dominio. El acoplamiento aferente (entrada) de model es alto: muchas clases dependen de él, lo cual es correcto porque es estable y no cambia. La regla sana es: las cosas estables (dominio) reciben muchas dependencias; las volátiles (I/O, arranque) dependen pero no son dependidas.

8. ¿Qué tipo de cohesión tiene Dial? ¿Y una clase “utilidad” como SecurityProtocols? Dial tiene cohesión **funcional**: todo lo que contiene colabora en una única tarea bien definida (la aritmética circular). SecurityProtocols tiene cohesión más débil, de tipo **lógica/utilitaria**: agrupa constantes relacionadas por tema (reglas) pero que no colaboran entre sí. Es aceptable para un catálogo de estrategias, pero no abusaría de clases utilidad porque tienden a convertirse en cajones de sastre.

9. Sin las capas, ¿qué problema concreto tendrías al testear? Si Safe leyera el fichero y parseara el texto, para probar la lógica de puntuación tendría que crear ficheros de prueba y depender del sistema de archivos: tests lentos y frágiles. Con las capas, el test instancia Safe/Engine con la estrategia que quiera y le pasa datos en memoria;

pruebo la lógica del dominio aislada, sin I/O. Eso es consecuencia directa del DIP y del SRP.

10. ¿Dónde está el polimorfismo y de qué tipo? Polimorfismo de subtipo en tiempo de ejecución: Safe invoca `protocol.calculatePoints(...)` y, según qué implementación inyecté (PART_A o PART_B), se ejecuta un comportamiento u otro, sin que Safe sepa cuál. La misma llamada produce resultados distintos según el objeto real detrás de la interfaz.

11. “Tell, Don’t Ask”: pruébame y dime dónde casi lo incumples. Lo cumplo en `safe.apply(instruction)`: le ordeno a la caja fuerte que actúe, no le pregunto su estado para decidir desde fuera. Casi lo incumpliría con `getZeroCount()`: es un getter, un “ask”. Es aceptable porque al final necesito *leer* el resultado para imprimirlo, pero si en lugar de leer el contador tomara decisiones de negocio fuera de Safe basándome en él, ahí sí estaría rompiendo el principio.

12. ¿Qué es lo que NO mejorarías aunque pudieras, y por qué (KISS/YAGNI)? No introduciría una interfaz propia para las estrategias del día 2: ya tengo `LongPredicate`, que encaja perfecto; crear `IdValidator` sería duplicar una abstracción sin ganancia (YAGNI). Tampoco generalizaría el cargador a un framework de fuentes múltiples mientras solo lea de fichero: añadiría complejidad que hoy no uso (KISS). Aplico los principios donde resuelven un problema real, no por coleccionarlos.

Cierre: frase para abrir la defensa

“Cada día comparte toda la mecánica entre la Parte A y la Parte B; lo único que cambia es una regla. Ese hecho es el que guía todo el diseño: la regla se modela como una estrategia que se inyecta desde el Main, y las capas garantizan que añadir o cambiar reglas, o cambiar la fuente de datos, no obligue a tocar el núcleo.”